

```

(define divide-list
  (lambda (l)
    (let divide
      ((l 1)
       (p '())
       (d '())))
    (if (null? l)
        (cons '() d)
        (divide (cdr l) d (cons (car l) p))))))

(define merge
  (lambda (s1 s2 . pred?)
    (let ((<= (if (null? pred?)
                  <= (car pred?))))
      (let merge ((s1 s1)
                   (s2 s2))
        (cond ((null? s1) s2)
              ((null? s2) s1)
              ((<= (car s1) (car s2)) (cons (car s1) (merge (cdr s1) s2)))
              (else (cons (car s2) (merge s1 (cdr s2)))))))

(define merge-sort
  (lambda (l . pred?)
    (let ((<= (if (null? pred?)
                  <= (car pred?))))
      (if (or (null? l) (null? (cdr l)))
          l
          (let ((divided-list (divide-list l)))
            (merge (merge-sort (car divided-list) <=)
                    (merge-sort (cdr divided-list) <=) <=))))))

```

Introducción al cómputo paralelo

Implementación de
Métodos Computacionales



Tecnológico
de Monterrey



Cómputo concurrente y distribuido

- **Computo concurrente:** múltiples procesos, alternando ente ellos, solo un proceso se ejecuta a la vez en un único procesador. Al alternar rápidamente entre procesos, simula que ocurren al mismo tiempo.
- **Computo distribuido:** procesos que se ejecutan en múltiples equipos.



Cómputo paralelo

- Posible gracias a arquitecturas de los procesadores multinúcleo y dispositivos dedicados, como las GPU
- Cada núcleo del procesador es capaz de trabajar en una tarea distinta simultáneamente.
- A su vez cada núcleo puede manejar su propia concurrencia
- Procesos multithread, con ramas de un solo proceso en diferentes núcleos del procesador



Particionamiento

Particionando por subtarear:

Partes independientes del problema,
diferentes tareas

Ejemplo:

Un servidor básico, donde un hilo
escucha, a la espera de peticiones

Otros hilos para procesarlas

Particionando por los datos:

Una misma tarea, que debe
realizarse de la misma forma, sobre
un gran conjunto de datos

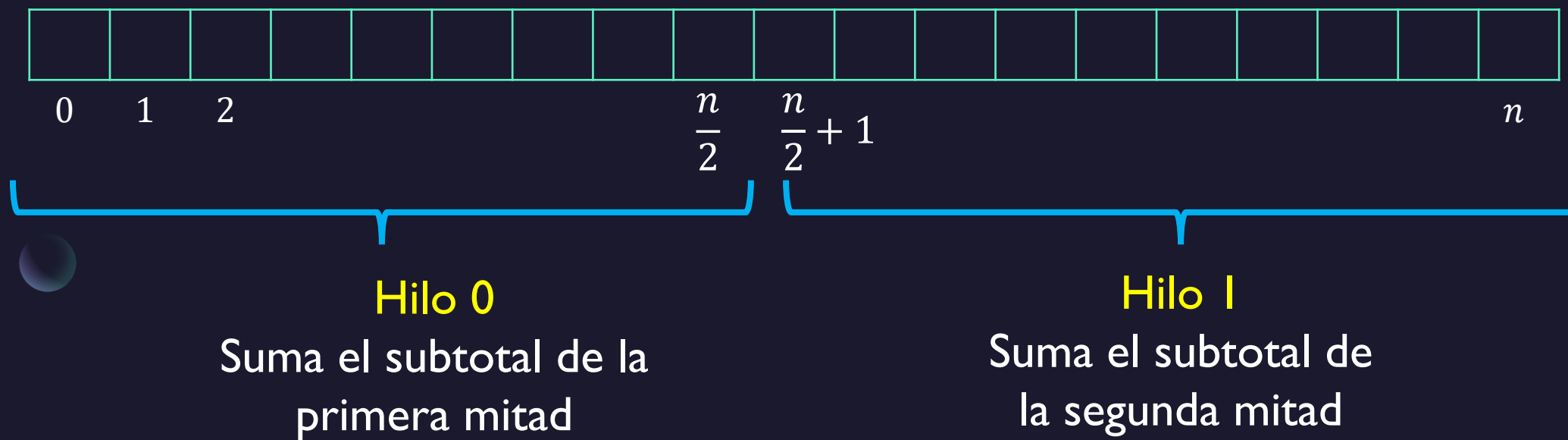
Ejemplo:

procesamiento de matrices,
renderizado de gráficos

En ambos casos la estrategia de resolución va ligada
con la naturaleza del problema

Particionando el problema

Supongamos que hay **2 hilos**. Podemos particionar los datos de la sig. forma



Chunk size: es el tamaño de la partición asociada a un hilo

Particionando el problema

Supongamos que hay 4 hilos



Cada hilo trabaja con
una cuarta parte de los
datos

¿Qué pasaría si hubiera
5, o 6 o x hilos?



Particionando el problema

¿Qué pasa cuando el número de datos
no es múltiplo del número de hilos?

Estrategia más simple:

el ultimo hilo toma la carga extra...

quedando **desbalanceado** respecto al resto





Particionando el problema

¿Qué pasa cuando el número de datos no es múltiplo del número de hilos?

Estrategia más simple:

el ultimo hilo toma la carga extra...
quedando **desbalanceado** respecto al resto



¿Por qué el desbalance es un problema?

Porque el tiempo total del cómputo depende del hilo que tarde mas



Particionando el problema



Algo mas elaborado:
Los primeros $\text{mod}(n, 4)$
hilos toman $n/4 + 1$ datos

El ultimo hilo toma $n/4$

Sigue existiendo desbalanceo, pero la
diferencia del **chunk size** con este
método es máximo de uno



En Racket: practica el particionado

```
; lista de n numeros random
(define (lista-rands ndatos max-val)
  (build-list ndatos (lambda (_) (random max-val)) )
)
```

Función que crea listas de valores aleatorios

nueva instrucción: *build-list*

Digamos que hay **T = 4 hilos**...

```
(87 242 104 67 80 120 279 268 161 160 447 82)
(180 401 62 244 483 419 281 252 419 275 358)
(227 313 405 332 365 182 322 215 383 450 42 492 378 397 242)
```

Crea una función que divida las listas en **T** secciones balanceadas

Herramientas

take: toma los primeros x elementos de una lista

```
> (take lista1 3)
'(33 60 65)
```

drop: se queda con los restantes elementos

```
> (drop lista1 3)
'(143 223 154 13 417 56 250 238 229)
```



En Racket: practica el particionado

```
(define (spliter lista x)
  (cond
    [(<= (length lista) x) (list lista) ]
    [else (cons (take lista x)
                 (spliter (drop lista x) x)
                )]
  )
)
```

Caso base:

*Si la lista restante tiene x elementos o menos,
devuelve la lista*

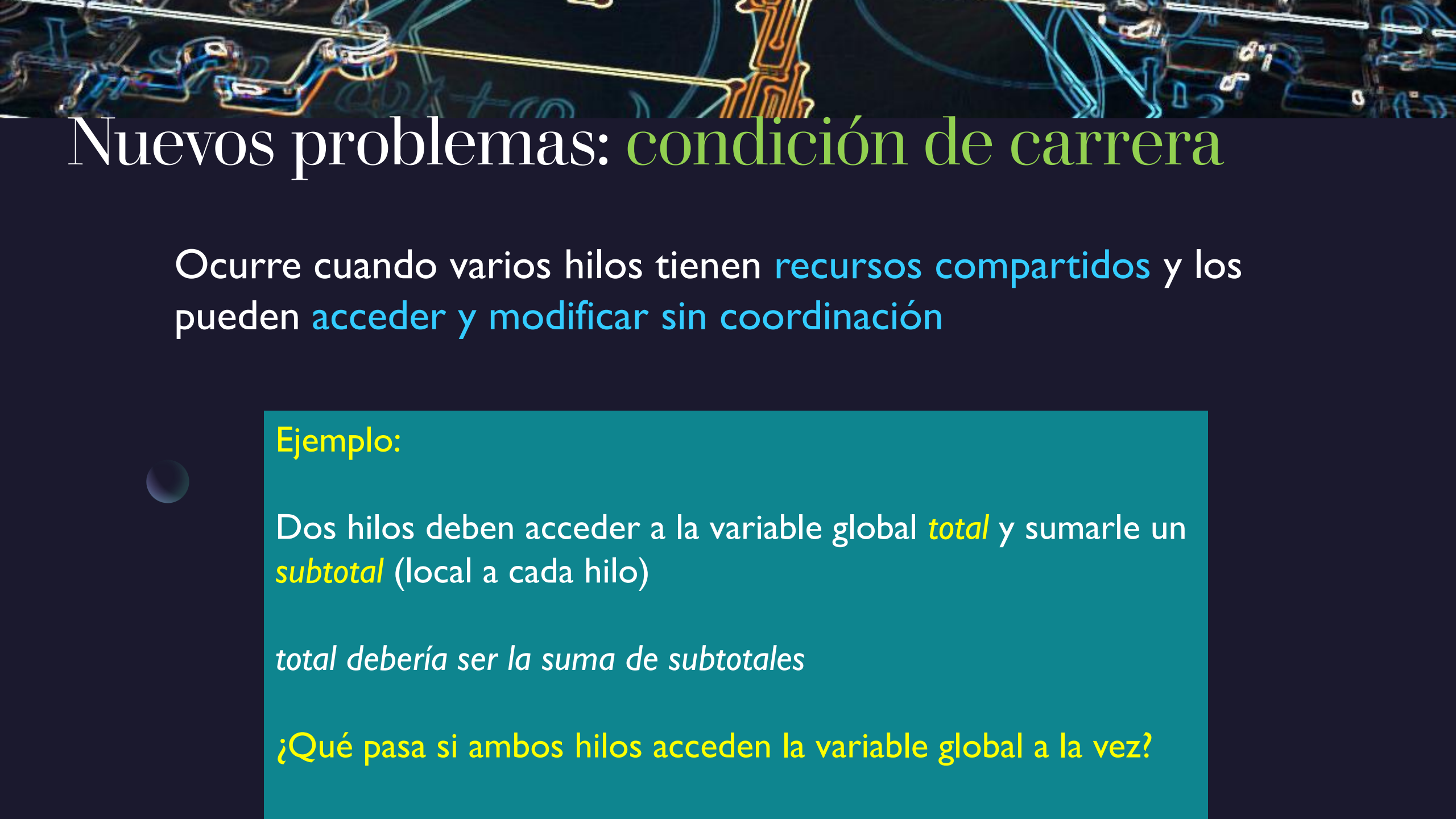
Caso recursivo:

Si no, toma x elementos

Llamada recursiva para el resto de la lista

```
> (spliter lista1 3)
'((222 352 91) (177 188 211) (300 435 93) (222 401 35))
```

```
(spliter lista2 3)
'((59 234 188) (395 171 464) (104 9 225) (138 351))
```



Nuevos problemas: condición de carrera

Ocurre cuando varios hilos tienen recursos compartidos y los pueden acceder y modificar sin coordinación

Ejemplo:

Dos hilos deben acceder a la variable global *total* y sumarle un *subtotal* (local a cada hilo)

total debería ser la suma de subtotales

¿Qué pasa si ambos hilos acceden la variable global a la vez?



Nuevos problemas: condición de carrera

Hilo A

Calcula localmente
subtotal = 24

En este momento
total = 0

total += *subtotal*

Hilo B

Calcula localmente
subtotal = 50

En este momento
total = 0

total += *subtotal*

- Supongamos que *subtotal* vale 24 en el hilo A y 50 en el hilo B
- Ambos hilos accedieron a *total* cuando su valor era 0
- Hilo A hace la suma y escribe que *total* es 24
- Hilo B hace la suma y escribe que *total* es 50

¿Cuál sería el valor al final?



Nuevos problemas: condición de carrera

Hilo A

Calcula localmente
subtotal = 24

En este momento
total = 0

total += *subtotal*

Hilo B

Calcula localmente
subtotal = 50

En este momento
total = 0

total += *subtotal*

- Supongamos que *subtotal* vale 24 en el hilo A y 50 en el hilo B
- Ambos hilos accedieron a *total* cuando su valor era 0
- Hilo A hace la suma y escribe que *total* es 24
- Hilo B hace la suma y escribe que *total* es 50

¿Cuál sería el valor al final? **Impredecible**



Nuevos problemas: condición de carrera

¿Cómo se evitan las condiciones de carrera?

Mutex (exclusión mutua)

Un mecanismo tipo candado para bloquear un recurso y que solo un hilo pueda acceder a la vez

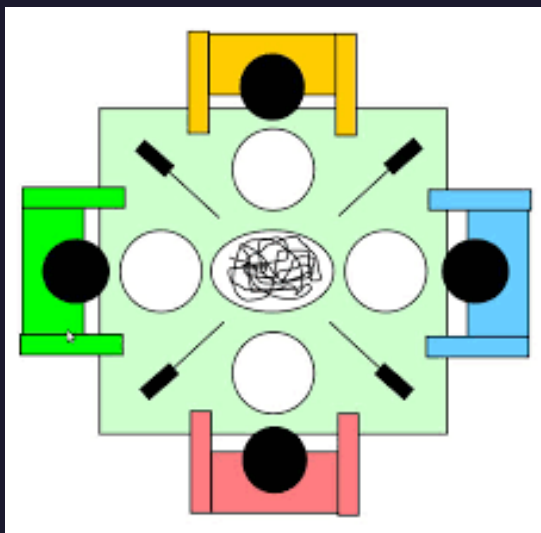
En este ejemplo, lock es un mutex
Pthreads (C++) implementa funciones lock / unlock

```
// Exclusion mutua usando mutex
pthread_mutex_lock(&lock);
    totalP += subtotal;
pthread_mutex_unlock(&lock);
```

Nuevos problemas: deadlock

La cena de los filósofos

Pasa cuando los hilos deben compartir recursos, pero caen en un bucle infinito de bloqueo y espera



Cena de los filósofos

- Cada filósofo necesita dos cubiertos para comer, pero no hay suficientes
- Cada filósofo toma el cubierto a su derecha (lo bloquea).
- El cubierto a su izquierda está bloqueado, así que espera
- Pero los demás también están esperando, para siempre...
- Nadie come, y mueren de hambre ☹️. **Investiga como evitarlo**

Midiendo el desempeño

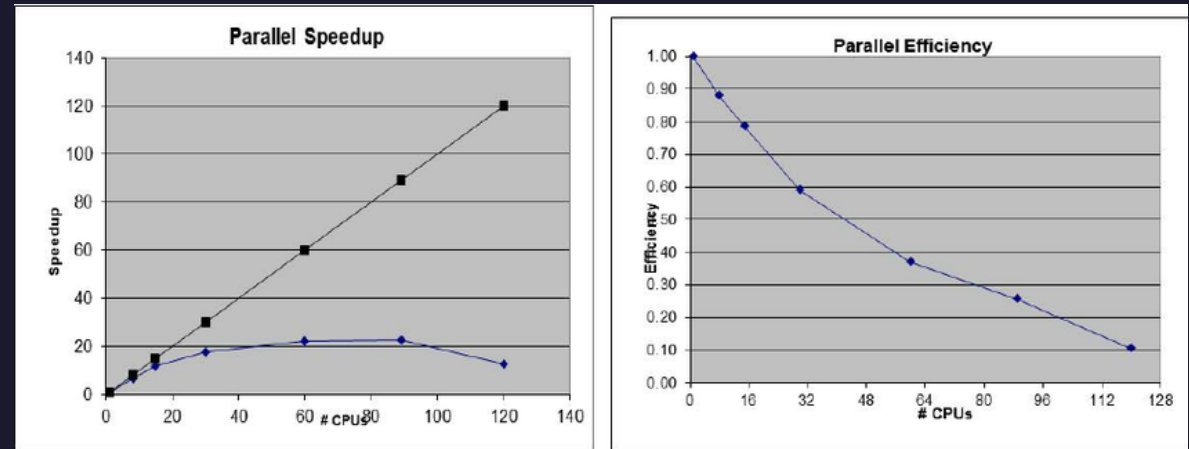
Speed-up (aceleración)

Proporción del tiempo secuencial y el paralelo

$$S = \frac{t(s)}{t(p)}$$

Eficiencia: El speed-up entre el número de procesadores

$$\frac{S}{cpus}$$



El speed-up ganado con cada hilo adicional decrece, eventualmente se estanca y luego cae

¿Por qué?

- Overhead: la creación, sincronización y manejo de los hilos también es costosa
- Cuando el número de hilos supera al número de procesadores, los hilos deben ser simulados (conurrencia)

Imágenes en Racket

Hay dos opciones de librerías:

2htdp/image (alto nivel pero lenta)

```
(require 2htdp/image)

; lee una imagen y crea mapa de bits
(define img (bitmap/file "cat.png"))
img

; obtener la altura
(define w (image-width img) )
(define h (image-height img) )

; convierte mapa de bits a lista de colores
; unidimensional en RGBA (son objetos)
(define pixels (image->color-list img) )

; el primer pixel
(define pl (car pixels) )
pl
```



racket/draw (básica pero rápida)

```
(require racket/draw)

; abre image, regresa argb
(define bmp (make-object bitmap% "cat.png"))
bmp

; ancho y altura
(define w (send bmp get-width))
(define h (send bmp get-height))

; prepara pixeles
(define pixel-bytes (make-bytes (* w h 4)))

; extrae pixeles
(send bmp get-argb-pixels 0 0 w h pixel-bytes)

; lista plana en ARGB
(define pixels (bytes->list pixel-bytes) )
(take pixels 4) ; primer pixel
```



Prueba con la imagen “*new-york-large.jpg*”

Medir el tiempo en Racket

Opción *time*:

Sintaxis *time* (*task*)

Solo imprime, sin
devolver resultado

No puede tener *define*
dentro, pero puede estar
dentro del *define*

o también:

```
(time  
  (begin  
    tarea1  
    tarea2  
  )  
)
```

Este es el tiempo que toma convertir el mapa de bits a objetos de clase color

```
> (define pixels (time (image->color-list img)))  
cpu time: 0 real time: 1 gc time: 0
```

Tres partes

cpu time: el tiempo usado en el cpu
real time: tiempo real, en reloj de pared
gc time: tiempo usado en garbage collection

Medir el tiempo en Racket

Opción *current-inexact-milliseconds* :

Este es el tiempo que toma convertir el mapa de bits a objetos de clase *color*, medido con *current-inexact-milliseconds*

El tiempo antes

```
(define start-seq (current-inexact-milliseconds))
```

```
(define pixels (time (image->color-list img)) ) ; task
```

Diferencia en tiempo
antes y después

```
(define end-seq (current-inexact-milliseconds))  
(displayln (format "\nTime: ~a ms\n" (- end-seq start-seq) ) )
```

Salidas: *time* vs *current-inexact-milliseconds*

```
cpu time: 0 real time: 1 gc time: 0
```

```
tiempo: 1.321044921875 ms
```


Imágenes en Racket

Prueba con la imagen “new-york-large.jpg”
¿Cuál tarda mas en obtener la listas de colores / pixeles?

2htdp/image (alto nivel)

```
(define start-seq (current-inexact-milliseconds))  
; lee una imagen y crea mapa de bits  
(define img (bitmap/file "new-york-large.jpg"))  
img  
  
; obtener la altura  
(define w (image-width img) )  
(define h (image-height img) )  
  
; convierte mapa de bits a lista de colores  
; unidimensional en RGBA (son objetos)  
(define pixels (time (image->color-list img)) )  
  
; (define pixels (time (image->color-list img)) ) ; task  
  
(define end-seq (current-inexact-milliseconds))  
(displayln (format "\ntiempo: ~a ms\n" (- end-seq start-seq) ) )
```

racket/draw (básica)

```
(define start-seq (current-inexact-milliseconds))  
; abre image, regresa argb  
(define bmp (make-object bitmap% "cat.png"))  
bmp  
  
; ancho y altura  
(define w (send bmp get-width))  
(define h (send bmp get-height))  
  
; prepara pixeles  
(define pixel-bytes (make-bytes (* w h 4)))  
  
; extrae pixeles  
(send bmp get-argb-pixels 0 0 w h pixel-bytes)  
  
; lista plana en ARGB  
(define pixels (bytes->list pixel-bytes) )  
  
(define end-seq (current-inexact-milliseconds))  
(displayln (format "\ntiempo: ~a ms\n" (- end-seq start-seq) ) )
```

Secuencial vs paralelo

Versión secuencial



Imagen original



Imagen filtrada completa

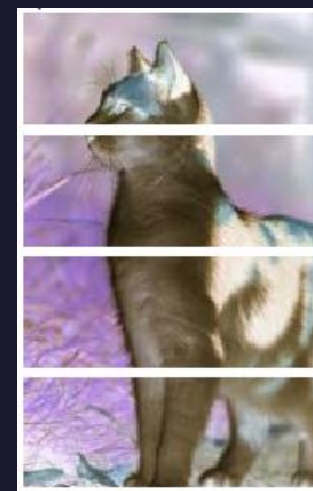
Filtro *invert*: $[A \ (255-R) \ (255-G) \ (255-B)]$
en cada pixel, alpha se mantiene, los valores RGB se invierten



Divide en T
chunks



Cada hilo
procesa una
sección



Une las
secciones



Versión paralela
con T = 4 hilos

Prueba e investiga para la SP

Estos son los primeros 4 pixeles de *cat.png*
(obtenidos con *racquet/draw* y *bytes→list*)

```
; lista plana en ARGB  
(define pixels (bytes→list pixel-bytes) )
```

```
' (255 60 78 66 255 57 75 63 255 57 75 61 255 55 74 58)
```

¿Cómo dividir estos datos en $T = 4$ secciones?

Pista: las filas de pixeles deben estar completas

¿Cómo aplicarías el filtro *invert* a una lista de pixeles dada?

Observa que para cada valor *rgb* la resta es la misma, y *alpha* queda igual

Filtro *invert*: $[A \ (255-R) \ (255-G) \ (255-B)]$
en cada pixel, *alpha* se mantiene, los valores RGB se invierten

Responde:

- ¿Es eficiente usar objetos de clase *color* (*2htdp/image*)? ¿Y el uso de listas y recursión?
- ¿Qué tan lentos son *append* vs *cons*, y *take/drop* vs *car /cdr*?
- ¿Cómo dividir los pixeles en chunks de tamaño similar según la cantidad de hilos?
- ¿Cómo medir tiempos?
- ¿Cómo hacer el proceso mas rápido?

¿Es viable usar listas e inmutabilidad en esta aplicación?
¿Que usar si no?

En esta SP puedes explorar opciones que no son estrictamente funcionales. Justifica su uso con evidencias (comparativas de tiempos)

¿Cuándo **paralelizar**?

- Cuando hay **poca interdependencia** entre datos, o entre las fases de un proceso
- Cuando se pueden realizar las **mismas operaciones** en cada hilo, y lo que varía son solo los datos
- Cuando el **problema es repetitivo** y lo suficientemente pesado para compensar el **overhead**: crear los chunks, gestionar los hilos, reunir el resultado, garbage collection... todo cuesta



El tipo de problema ideal para GPUs, por ejemplo, el renderizado de gráficos con iluminación, viento, lluvia, etc.